

// Day 16 of 30

SYSTEM DESIGN BREAKDOWN

How Zomato Handles 1M Orders Per Day

→ Swipe through all 6 slides



Shubham Bhati

[linkedin.com/in/bhatishubham](https://www.linkedin.com/in/bhatishubham)

1/8

The Scale

Numbers that matter

Daily orders: 1,000,000+

Restaurants: 300,000+

Concurrent users: millions

Delivery partners: 350,000+

Peak hours (12-2 PM):

→ 10x normal traffic

→ 50,000+ orders/hour

→ Real-time tracking for ALL

💡 Scale changes everything about architecture



Shubham Bhati

[linkedin.com/in/bhatishubham](https://www.linkedin.com/in/bhatishubham)

Microservices Architecture

Not one app. Many small services.

User Service → auth, profiles

Restaurant Service → menus, hours

Order Service → place, track, cancel

Payment Service → transactions

Delivery Service → partner matching

Notification → push, SMS, email

Search → 'biryani near me'

Each service: own DB, own team, own deploy

💡 Each service scales independently



Database Strategy

Right database for right job

MySQL/PostgreSQL:

→ Orders, payments (ACID needed)

MongoDB:

→ Menus, reviews (flexible schema)

Redis:

→ Sessions, menus cache (fast reads)

→ Reduces DB load by 80%

ElasticSearch:

→ 'biryani near me' in 50ms

💡 Polyglot persistence = use the right tool



Shubham Bhati

linkedin.com/in/bhatishubham

Message Queues

Async = fast + resilient

Order placed → Event published

Kafka consumers:

- Payment service processes payment
- Restaurant gets notification
- Delivery partner assigned
- User gets push notification

If notification service is slow?

- Order service doesn't care
- Decoupled. Fast. Reliable.

💡 Async messaging = backbone of microservices



Shubham Bhati

[linkedin.com/in/bhatishubham](https://www.linkedin.com/in/bhatishubham)

Payment — Hardest Part

Money can never go wrong

Idempotency key:

→ Pay twice? Same result. No double charge.

Pessimistic locking:

→ Lock wallet row during transaction

Saga pattern:

→ Payment fails? → Cancel order → Refund

Retry + Dead letter queue:

→ Failed payments retried 3x

→ Then moved to DLQ for manual review

💡 Payment systems need ALL of these



Shubham Bhati

linkedin.com/in/bhatishubham

Key Takeaways

Apply these in any backend system

1. Microservices > Monolith at scale
2. Async > Sync for non-critical paths
3. Cache everything (Redis)
4. Idempotency is mandatory
5. Each service owns its database
6. Design for failure (circuit breaker)
7. Message queues decouple everything

💡 These principles = system design interviews



Shubham Bhati

linkedin.com/in/bhatishubham

done = true

Follow for more!

Like · Comment · Repost

Shubham Bhati

[linkedin.com/in/bhatishubham](https://www.linkedin.com/in/bhatishubham)



Shubham Bhati

[linkedin.com/in/bhatishubham](https://www.linkedin.com/in/bhatishubham)